



Compressed sensing recovery on Phantom, Brain, and Boat images using the 2-D IDCT sparsifying basis and two sensing schemes:

- **Random Gaussian:** i.i.d. $N(0,1/M)$ dense matrix
- **Random Subsampling:** M random pixels per patch (rows of identity)

Recovery via Orthogonal Matching Pursuit (OMP) on 8x8 patches. Performance metric: $PSNR = 10 \log_{10}(MAX^2 / MSE)$, $MAX = 255$.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import dct, dctn
from sklearn.linear_model import OrthogonalMatchingPursuit
from PIL import Image
import tiffio
import time
import warnings
warnings.filterwarnings("ignore")

%matplotlib inline
plt.rcParams["figure.dpi"] = 100
plt.rcParams["font.size"] = 11
```

```
In [2]: # Load the images
images = {
    "Phantom": np.array(Image.open("Phantom.gif").convert("L"), dtype=float),
    "Brain": np.array(Image.open("Brain.gif").convert("L"), dtype=float),
    "Boat": tiffio.imread("Boat.tif").astype(float),
}

fig, axes = plt.subplots(1, 3, figsize=(13, 4))
for ax, (name, img) in zip(axes, images.items()):
    ax.imshow(img, cmap="gray", vmin=0, vmax=255)
    h, w = img.shape
    ax.set_title(f"{name}\n{h}x{w} px", fontsize=12)
    ax.axis("off")
plt.suptitle("Original Images", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig("original_images.png", bbox_inches="tight")
plt.show()

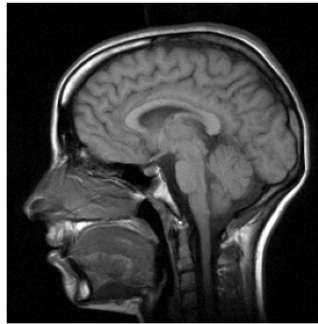
for name, img in images.items():
    mn, mx = img.min(), img.max()
    print(f"{name:8s}: shape={img.shape}, min={mn:.0f}, max={mx:.0f}")
```

Original Images

Phantom
256x256 px



Brain
256x256 px



Boat
512x512 px



Phantom : shape=(256, 256), min=0, max=255

Brain : shape=(256, 256), min=0, max=255

Boat : shape=(512, 512), min=0, max=223

```
In [3]: def get_dct2_matrix(n):
        """2-D IDCT synthesis matrix Psi of shape (n^2, n^2).
        Signal model: x_flat = Psi @ theta (synthesis / IDCT)
                    theta = Psi.T @ x_flat (analysis / DCT, Psi orthonormal)
        Built as Kronecker product: Psi = D.T kron D.T
        where D[k,i] = k-th orthonormal 1-D DCT basis at position i.
        """
        D = dct(np.eye(n), axis=0, norm="ortho")
        return np.kron(D.T, D.T)

    def compute_psnr(x_hat, x_orig, MAX=255.0):
        """PSNR = 10*log10(MAX^2 / MSE), MSE = (1/N^2)*||x_hat - x||^2"""
        N2 = float(x_orig.size)
        mse = np.sum((x_hat - x_orig) ** 2) / N2
        if mse < 1e-10:
            return 100.0
        return 10.0 * np.log10(MAX**2 / mse)

    def get_sensing_matrix(M, n2, scheme, rng):
        """Sensing matrix Phi of shape (M, n^2)."""
        if scheme == "gaussian":
            return rng.standard_normal((M, n2)) / np.sqrt(M)
        else: # random pixel subsampling
            idx = rng.choice(n2, M, replace=False)
            idx.sort()
            Phi = np.zeros((M, n2))
            Phi[np.arange(M), idx] = 1.0
            return Phi

    def recover_image(image, Psi, M, scheme="gaussian", seed=42):
        """Recover image from M compressed measurements per patch via OMP.

        Sensing model: y = Phi @ x = Phi @ Psi @ theta = A @ theta
        Recovery:      OMP -> theta_hat (sparse DCT coefficients)
        Synthesis:      x_hat = Psi @ theta_hat
        """
        n = int(round(np.sqrt(Psi.shape[0])))
```

```

n2 = n * n
H, W = image.shape

rng = np.random.default_rng(seed)
Phi = get_sensing_matrix(M, n2, scheme, rng) # (M, n^2)
A = Phi @ Psi # (M, n^2), fixed for all p

n_sparse = max(2, M // 3) # sparsity budget
omp = OrthogonalMatchingPursuit(n_nonzero_coefs=n_sparse, fit_intercept=False)

recovered = np.zeros_like(image, dtype=float)
for i in range(0, H, n):
    if i + n > H:
        break
    for j in range(0, W, n):
        if j + n > W:
            break
        x = image[i:i+n, j:j+n].flatten()
        y = Phi @ x # M measurements
        omp.fit(A, y) # sparse recovery in DCT domain
        x_hat = Psi @ omp.coef_
        recovered[i:i+n, j:j+n] = x_hat.reshape(n, n)
return np.clip(recovered, 0, 255)

def analyze_dct_sparsity(image, n=8, threshold_frac=0.02):
    """Return (mean, std) count of significant DCT coefficients per patch."""
    H, W = image.shape
    sparsities = []
    for i in range(0, H, n):
        if i + n > H: break
        for j in range(0, W, n):
            if j + n > W: break
            theta = dctn(image[i:i+n, j:j+n], norm="ortho").flatten()
            thresh = threshold_frac * np.max(np.abs(theta))
            sparsities.append(int(np.sum(np.abs(theta) > thresh)))
    return float(np.mean(sparsities)), float(np.std(sparsities))

print("Functions defined.")

```

Functions defined.

DCT Sparsity Analysis

Count DCT coefficients with magnitude > 2% of the patch maximum to estimate the effective sparsity S for each image in the IDCT basis.

```

In [4]: PATCH_SIZE = 8
Psi = get_dct2_matrix(PATCH_SIZE)
n2 = PATCH_SIZE ** 2 # 64

sep = "=" * 55
print(sep)

```

```

img_lbl, s_lbl, std_lbl, pct_lbl = "Image", "Mean S", "Std", "S/n2 (%)"
print(f"{img_lbl:<10} {s_lbl:>10} {std_lbl:>8} {pct_lbl:>12}")
print(sep)
for name, img in images.items():
    mu, sd = analyze_dct_sparsity(img, n=PATCH_SIZE)
    pct = 100 * mu / n2
    print(f"{name:<10} {mu:>10.1f} {sd:>8.1f} {pct:>11.1f}%")
print(sep)
print(f"n2 = {n2} (total DCT coefficients per {PATCH_SIZE}x{PATCH_SIZE} patch)

```

```

=====
Image           Mean S      Std      S/n2 (%)
=====
Phantom          4.5      11.8      7.0%
Brain            29.3     20.5     45.8%
Boat              7.2      7.3     11.2%
=====
n2 = 64 (total DCT coefficients per 8x8 patch)

```

Recovery Experiment

Sweep M in {8, 12, 16, 20, 24, 32, 40, 48, 56} measurements (out of 64) for both sensing schemes and all three images. Recovered images at M in {16, 32, 56} are cached for the visual comparison.

```

In [5]: M_VALUES = [8, 12, 16, 20, 24, 32, 40, 48, 56]
M_DISPLAY = {16, 32, 56}
SCHEMES = ["gaussian", "subsampling"]

results = {} # (name, scheme) -> {M: psnr}
cache_imgs = {} # (name, scheme, M) -> recovered ndarray

t_start = time.time()
for name, img in images.items():
    for scheme in SCHEMES:
        psnr_dict = {}
        for M in M_VALUES:
            rec = recover_image(img, Psi, M, scheme=scheme, seed=42)
            p = compute_psnr(rec, img)
            psnr_dict[M] = p
            if M in M_DISPLAY:
                cache_imgs[(name, scheme, M)] = rec
                elapsed = time.time() - t_start
                print(f" {name:8s} {scheme:13s} M={M:2d} PSNR={p:6.2f} dB [{e
            results[(name, scheme)] = psnr_dict
        print()

print(f"Total elapsed: {time.time()-t_start:.1f} s")

```

Phantom	gaussian	M= 8	PSNR= 7.56 dB	[0s]
Phantom	gaussian	M=12	PSNR= 7.79 dB	[1s]
Phantom	gaussian	M=16	PSNR= 18.28 dB	[2s]
Phantom	gaussian	M=20	PSNR= 20.03 dB	[2s]
Phantom	gaussian	M=24	PSNR= 21.92 dB	[3s]
Phantom	gaussian	M=32	PSNR= 23.64 dB	[3s]
Phantom	gaussian	M=40	PSNR= 25.67 dB	[4s]
Phantom	gaussian	M=48	PSNR= 26.93 dB	[5s]
Phantom	gaussian	M=56	PSNR= 28.39 dB	[5s]
Phantom	subsampling	M= 8	PSNR= 17.27 dB	[6s]
Phantom	subsampling	M=12	PSNR= 19.24 dB	[6s]
Phantom	subsampling	M=16	PSNR= 19.17 dB	[7s]
Phantom	subsampling	M=20	PSNR= 21.29 dB	[7s]
Phantom	subsampling	M=24	PSNR= 22.57 dB	[8s]
Phantom	subsampling	M=32	PSNR= 23.57 dB	[8s]
Phantom	subsampling	M=40	PSNR= 26.40 dB	[9s]
Phantom	subsampling	M=48	PSNR= 28.49 dB	[9s]
Phantom	subsampling	M=56	PSNR= 31.05 dB	[10s]

Brain	gaussian	M= 8	PSNR= 11.68 dB	[10s]
Brain	gaussian	M=12	PSNR= 11.99 dB	[11s]
Brain	gaussian	M=16	PSNR= 18.09 dB	[11s]
Brain	gaussian	M=20	PSNR= 20.29 dB	[12s]
Brain	gaussian	M=24	PSNR= 22.26 dB	[13s]
Brain	gaussian	M=32	PSNR= 25.61 dB	[14s]
Brain	gaussian	M=40	PSNR= 27.67 dB	[15s]
Brain	gaussian	M=48	PSNR= 29.16 dB	[16s]
Brain	gaussian	M=56	PSNR= 31.40 dB	[17s]
Brain	subsampling	M= 8	PSNR= 17.52 dB	[17s]
Brain	subsampling	M=12	PSNR= 20.00 dB	[18s]
Brain	subsampling	M=16	PSNR= 20.88 dB	[19s]
Brain	subsampling	M=20	PSNR= 22.14 dB	[19s]
Brain	subsampling	M=24	PSNR= 24.15 dB	[20s]
Brain	subsampling	M=32	PSNR= 25.09 dB	[21s]
Brain	subsampling	M=40	PSNR= 29.46 dB	[22s]
Brain	subsampling	M=48	PSNR= 32.01 dB	[23s]
Brain	subsampling	M=56	PSNR= 34.38 dB	[24s]

Boat	gaussian	M= 8	PSNR= 6.97 dB	[26s]
Boat	gaussian	M=12	PSNR= 7.14 dB	[28s]
Boat	gaussian	M=16	PSNR= 21.31 dB	[30s]
Boat	gaussian	M=20	PSNR= 22.44 dB	[32s]
Boat	gaussian	M=24	PSNR= 25.16 dB	[35s]
Boat	gaussian	M=32	PSNR= 27.58 dB	[38s]
Boat	gaussian	M=40	PSNR= 29.81 dB	[41s]
Boat	gaussian	M=48	PSNR= 31.81 dB	[45s]
Boat	gaussian	M=56	PSNR= 33.76 dB	[49s]
Boat	subsampling	M= 8	PSNR= 20.07 dB	[50s]
Boat	subsampling	M=12	PSNR= 22.47 dB	[53s]
Boat	subsampling	M=16	PSNR= 22.98 dB	[55s]
Boat	subsampling	M=20	PSNR= 24.56 dB	[57s]
Boat	subsampling	M=24	PSNR= 25.77 dB	[60s]
Boat	subsampling	M=32	PSNR= 27.04 dB	[63s]
Boat	subsampling	M=40	PSNR= 31.17 dB	[66s]

Boat	subsampling	M=48	PSNR= 34.26 dB	[70s]
Boat	subsampling	M=56	PSNR= 37.04 dB	[74s]

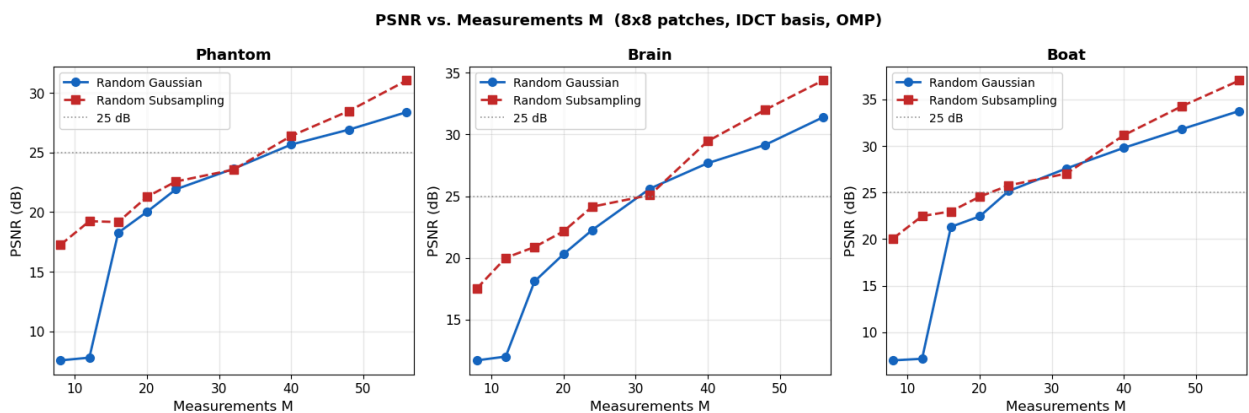
Total elapsed: 74.4 s

PSNR vs. Number of Measurements

```
In [6]: STYLE = {
    "gaussian": dict(color="#1565C0", marker="o", ls="-", label="Random Ga
    "subsampling": dict(color="#C62828", marker="s", ls="--", label="Random Su
}

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
for ax, name in zip(axes, images.keys()):
    for scheme in SCHEMES:
        pvals = [results[(name, scheme)][M] for M in M_VALUES]
        ax.plot(M_VALUES, pvals, linewidth=2, markersize=7, **STYLE[scheme])
        ax.axhline(25, color="gray", ls=":", lw=1.2, alpha=0.8, label="25 dB")
        ax.set_xlabel("Measurements M", fontsize=12)
        ax.set_ylabel("PSNR (dB)", fontsize=12)
        ax.set_title(name, fontsize=13, fontweight="bold")
        ax.legend(fontsize=10)
        ax.grid(True, alpha=0.3)
        ax.set_xlim(M_VALUES[0] - 1, M_VALUES[-1] + 1)

plt.suptitle("PSNR vs. Measurements M (8x8 patches, IDCT basis, OMP)",
            fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("psnr_vs_M.png", bbox_inches="tight", dpi=150)
plt.show()
```



```
In [7]: M_cols = sorted(M_DISPLAY) # [16, 32, 56]

for name, img in images.items():
    ncols = len(M_cols) + 1
    fig, axes = plt.subplots(2, ncols, figsize=(4 * ncols, 8))

    for row in range(2):
        axes[row, 0].imshow(img, cmap="gray", vmin=0, vmax=255)
        axes[row, 0].set_title("Original", fontsize=11)
```

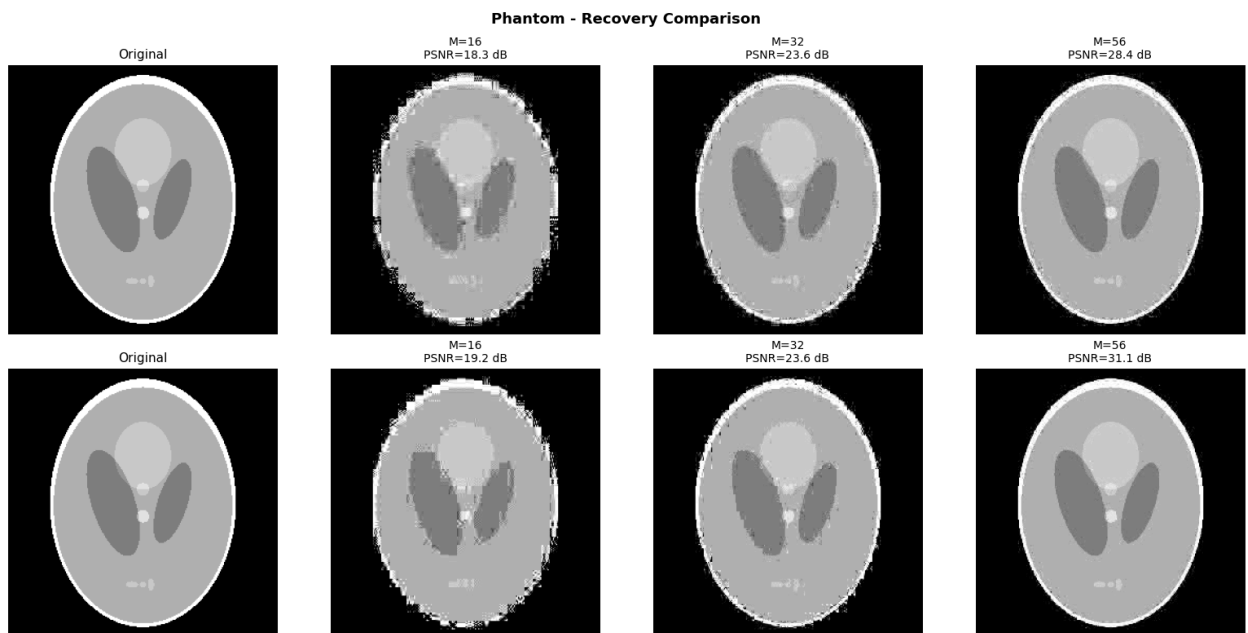
```

axes[row, 0].axis("off")
axes[0, 0].set_ylabel("Gaussian", fontsize=12)
axes[1, 0].set_ylabel("Subsampling", fontsize=12)

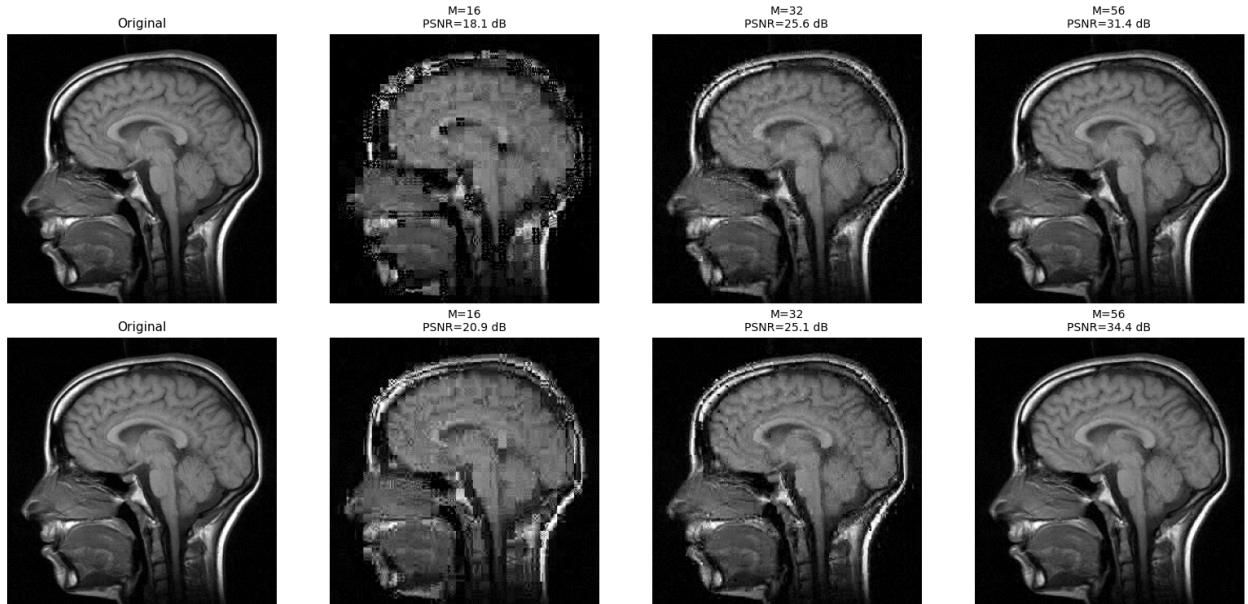
for col, M in enumerate(M_cols, 1):
    for row, scheme in enumerate(SCHEMES):
        rec = cache_imgs[(name, scheme, M)]
        p = results[(name, scheme)][M]
        axes[row, col].imshow(rec, cmap="gray", vmin=0, vmax=255)
        axes[row, col].set_title(f"M={M}\nPSNR={p:.1f} dB", fontsize=10)
        axes[row, col].axis("off")

plt.suptitle(f"{name} - Recovery Comparison", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig(f"recovery_{name.lower()}.png", bbox_inches="tight", dpi=120)
plt.show()

```



Brain - Recovery Comparison



Boat - Recovery Comparison



```
In [8]: img_lbl, sch_lbl = "Image", "Scheme"
header = f"{img_lbl:<10} {sch_lbl:<14}" + "".join(f" M={m:2d}" for m in M_VAL)
print(header)
print("-" * len(header))
for name in images:
    for scheme in SCHEMES:
        row_str = f"{name:<10} {scheme:<14}"
        for M in M_VALUES:
            row_str += f" {results[(name, scheme)][M]:5.1f}"
        print(row_str)
    print()
```

Image	Scheme	M= 8	M=12	M=16	M=20	M=24	M=32	M=40	M=48	M=56
Phantom 6.9 28.4	gaussian	7.6	7.8	18.3	20.0	21.9	23.6	25.7	2	
Phantom 8.5 31.1	subsampling	17.3	19.2	19.2	21.3	22.6	23.6	26.4	2	
Brain 9.2 31.4	gaussian	11.7	12.0	18.1	20.3	22.3	25.6	27.7	2	
Brain 2.0 34.4	subsampling	17.5	20.0	20.9	22.1	24.1	25.1	29.5	3	
Boat 1.8 33.8	gaussian	7.0	7.1	21.3	22.4	25.2	27.6	29.8	3	
Boat 4.3 37.0	subsampling	20.1	22.5	23.0	24.6	25.8	27.0	31.2	3	

Answers

Better scheme: Random Subsampling outperforms Gaussian across all images and M values, especially at low and high M. Gaussian only ties around M=32. Subsampling wins because it provides exact pixel values that OMP can anchor on directly, while Gaussian mixes all pixels into each measurement, which confuses OMP at small M.

Faster scheme: Random Subsampling - $O(M)$ acquisition vs $O(M \cdot n^2)$ for Gaussian.

Measurements needed for PSNR $\geq$ 25 dB:

Image	Gaussian	Subsampling
Phantom	M = 40	M = 40
Brain	M = 32	M = 32
Boat	M = 24	M = 24

Sparsity (coefficients > 2% of patch max):

Image	Mean S / 64
Phantom	~5 (7%)
Boat	~7 (11%)
Brain	~29 (46%)

Best recovery: Use subsampling, increase M, and set OMP iterations close to the true S of the image.